MACHINE LEARNING IN PHYSICS
GRAPH NEURAL NETWORKS

HARRISON B. PROSPER

PHY6937 / PHY4936

Outline

- Introduction
- Graph Neural Networks @ IceCube
- GNN Tutorial
- Summary

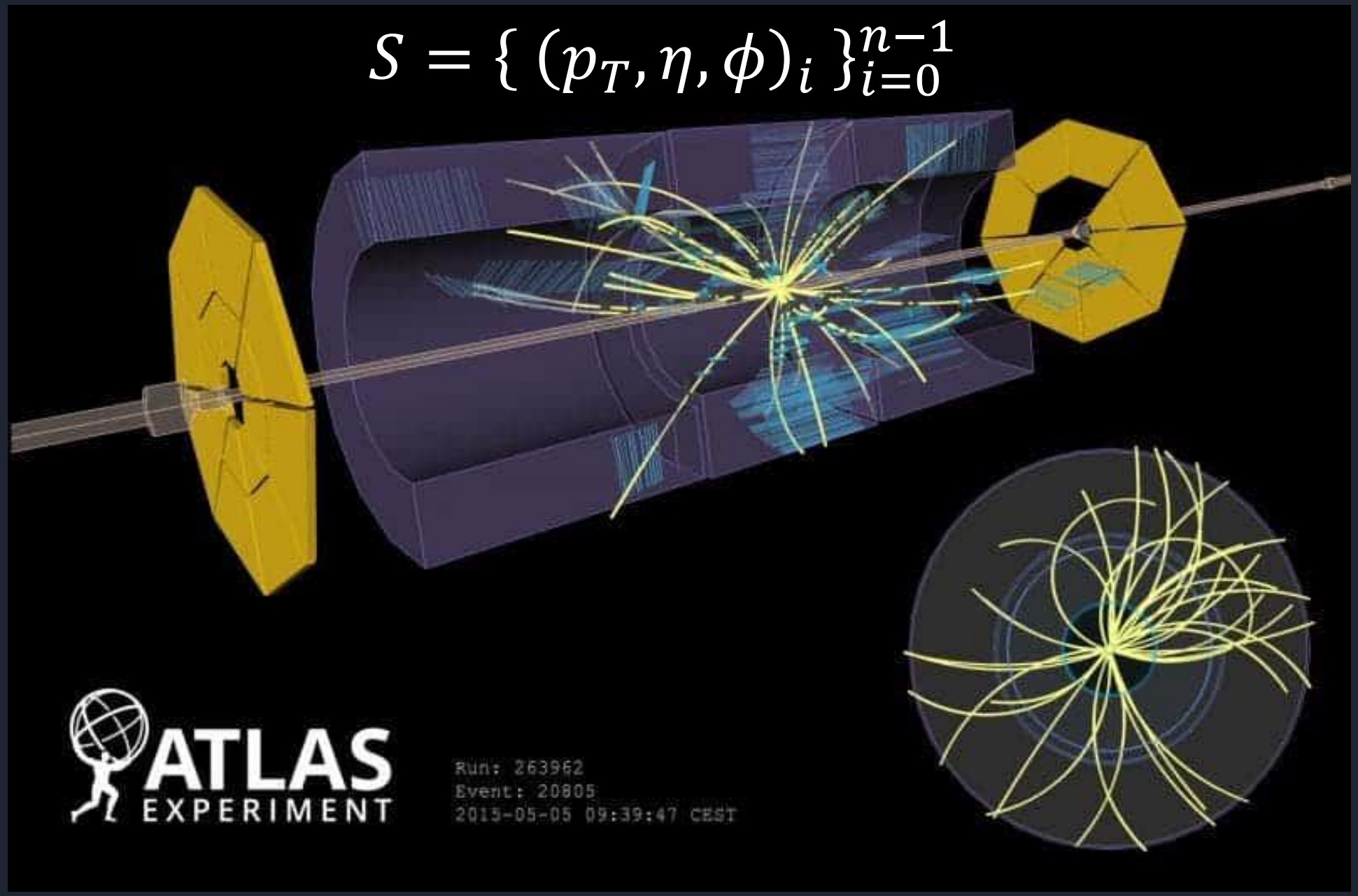
Introduction (1)

- Any sample $S = \{x_i\}_{i=0}^{n-1}$ of size n within a dataset can be regarded as a cloud of n points in a vector space, that is, as a **point cloud**.

- Standard neural networks expect the size of the point cloud S to be fixed for every sample.

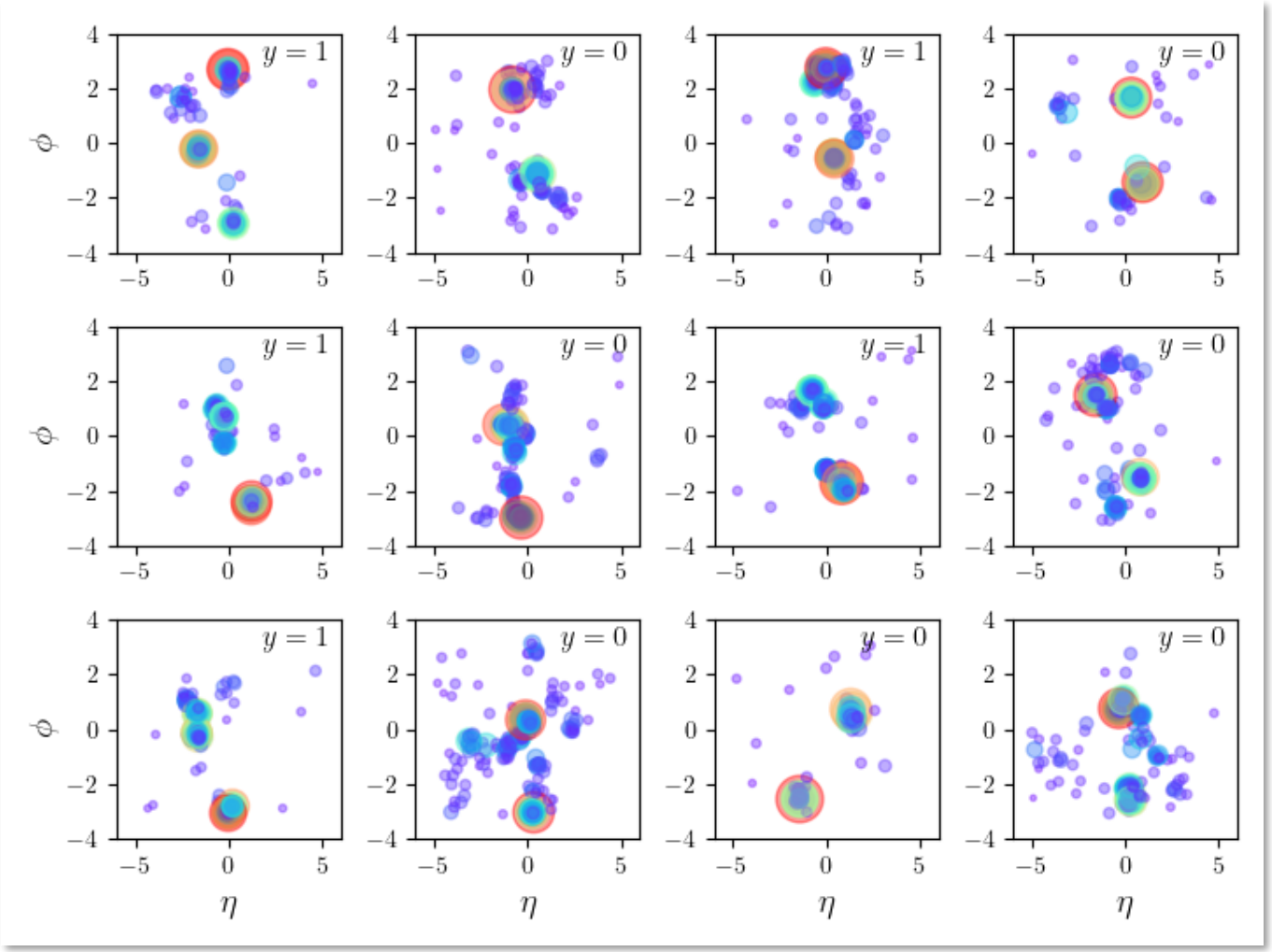
- However, there are datasets in which the size of the point cloud varies from one cloud to the next. Example: the number of particles created in high-energy particle collisions can vary from one collision event to the next.

$$S = \{ (p_T, \eta, \phi)_i \}_{i=0}^{n-1}$$



Introduction (2)

Particle collision events for two classes of interactions, labeled $y = 1$ and $y = 0$.

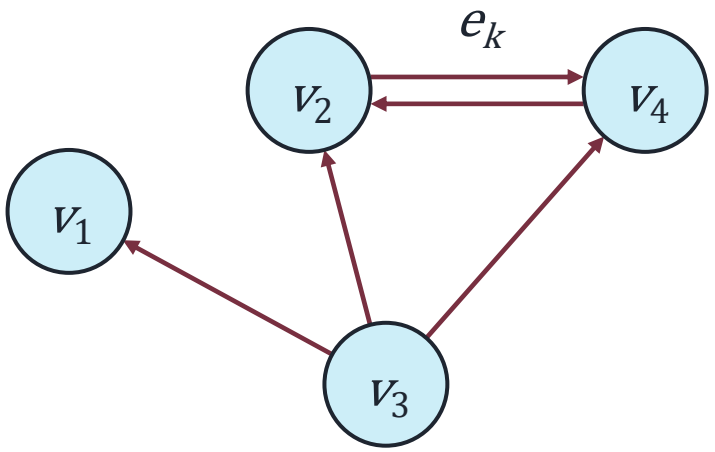


https://lhco2020.github.io/homepage/

Introduction (3)

- The data from a single collision event can be modeled as a point cloud with *n* points in a *d*-dimensional space.

- One way to represent a point cloud is as a graph.

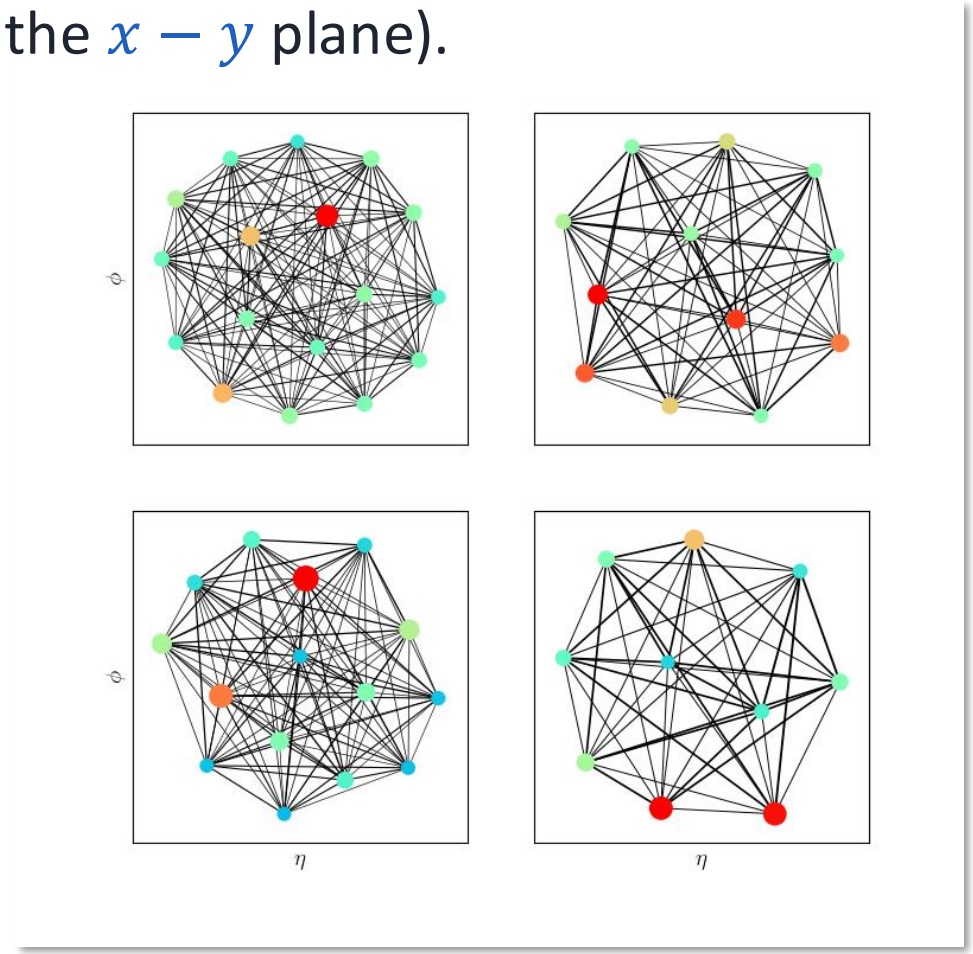


- A graph $G = (V, E)$ comprises vertices (or nodes) and edges, with attributes $V = \{v_i\}$ and $E = \{e_k\}$, respectively.

Example: Collision Events as Graphs

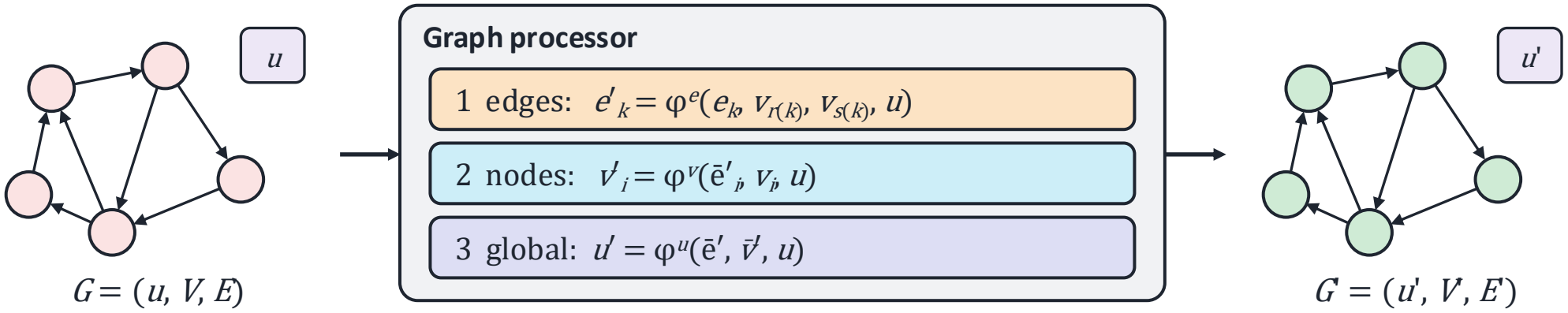
- The dots (i.e., nodes) represent particles, with the colors representing different transverse momenta (momentum in the $x - y$ plane).

- The lines (i.e., edges) represent the relationships between the particles, with the thickness of the lines representing the strength of the relationships.



Introduction (4)

A graph neural network is a function that operates on data represented as graphs.

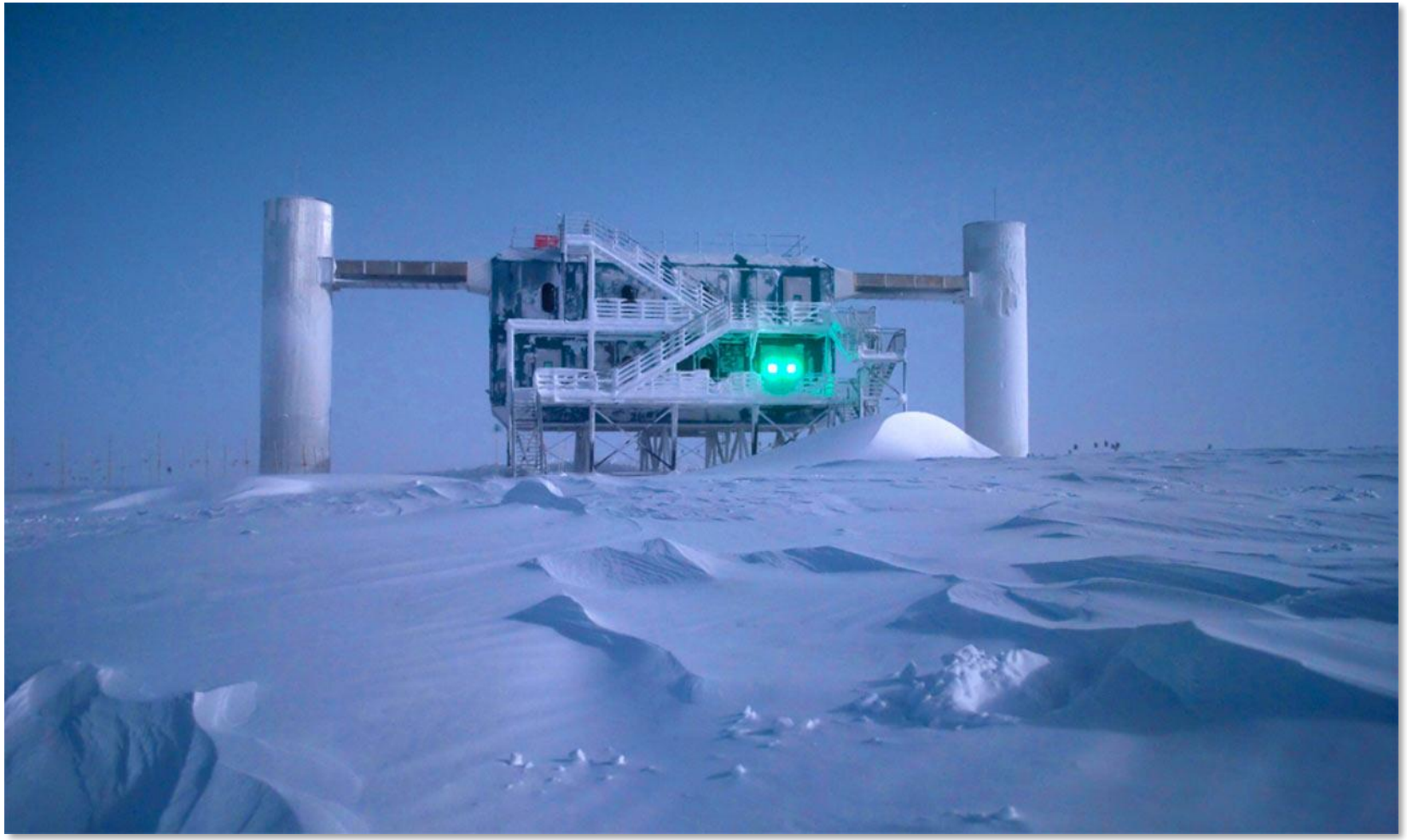


A graph enters a graph processor on the left and exits the processor on the right with altered graph attributes.

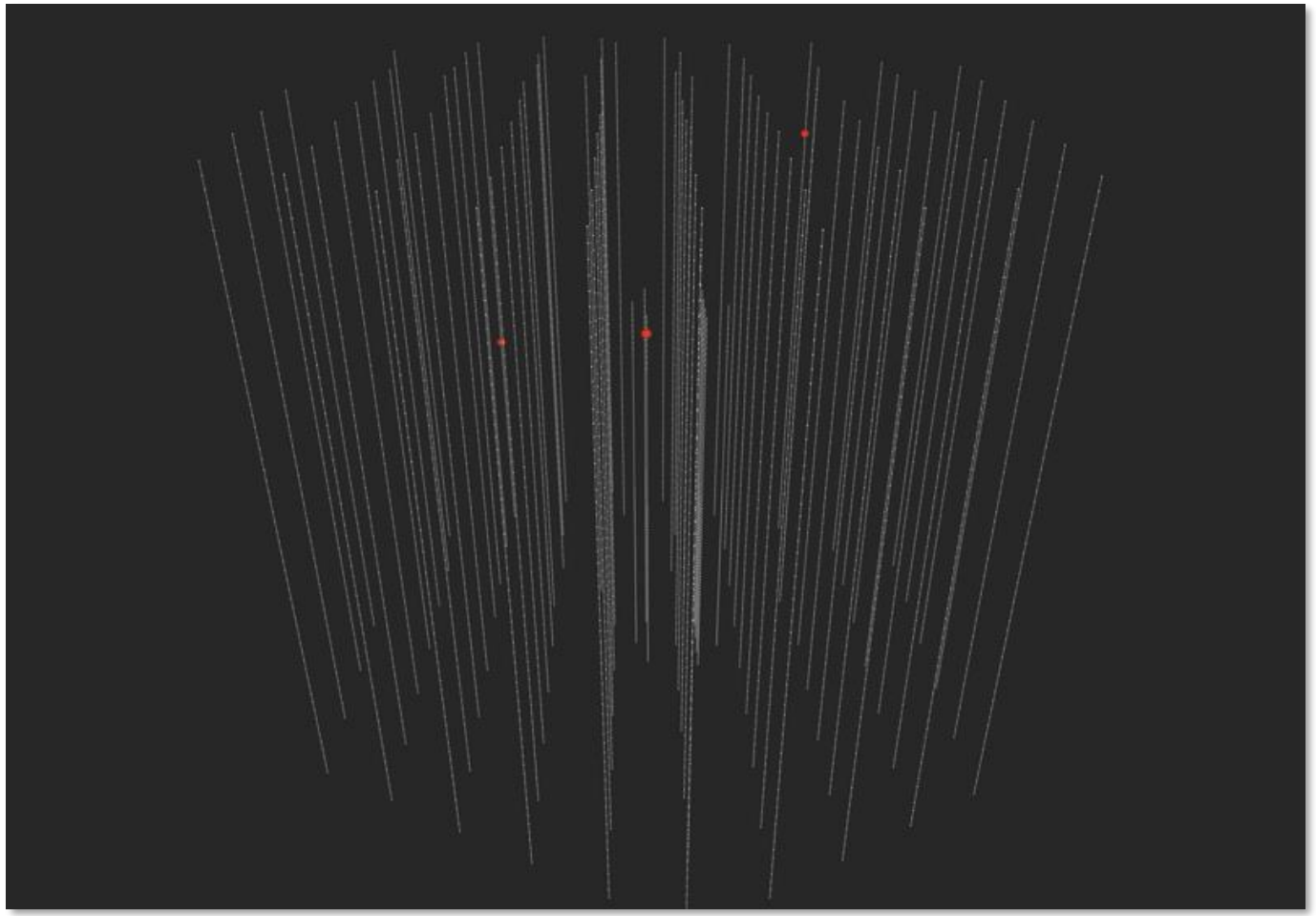
https://arxiv.org/abs/1806.01261

IceCube GNN

IceCube GNN (1)



IceCube GNN (2)



IceCube GNN: Data (1)

The data collected by the IceCube detector consists of signals from photomultipliers*.

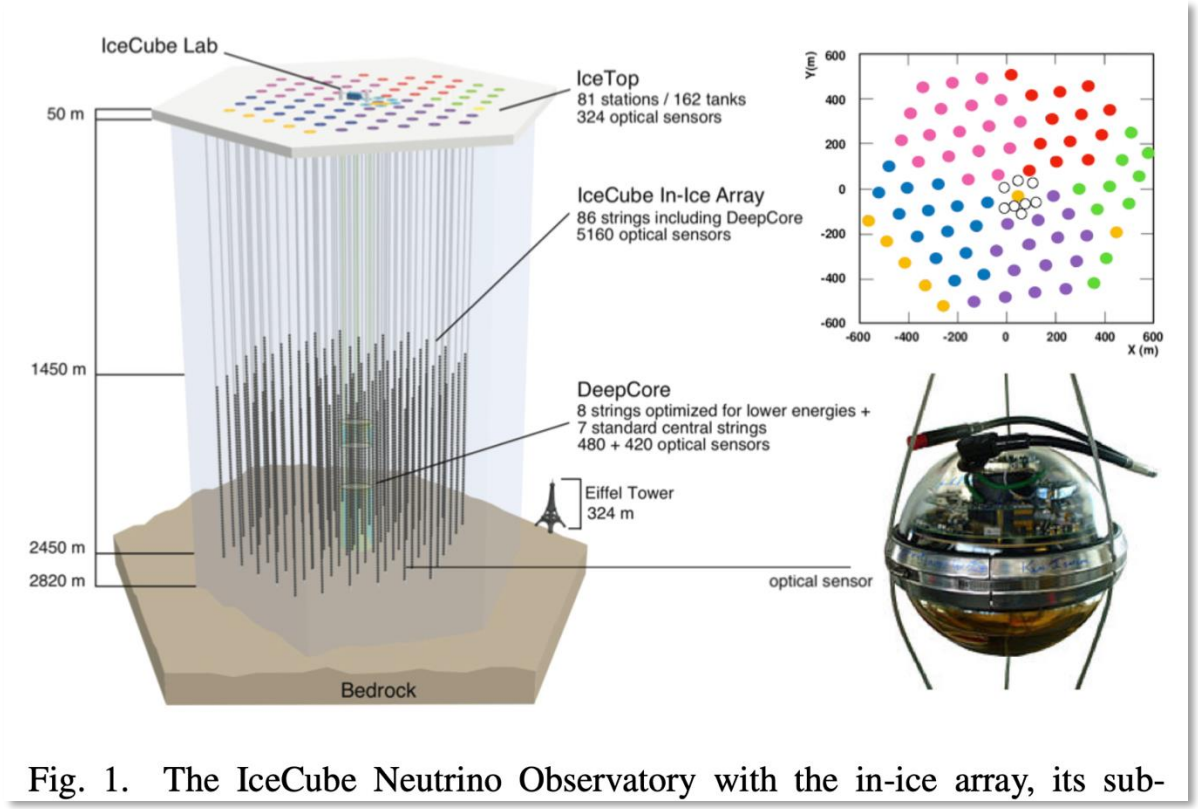


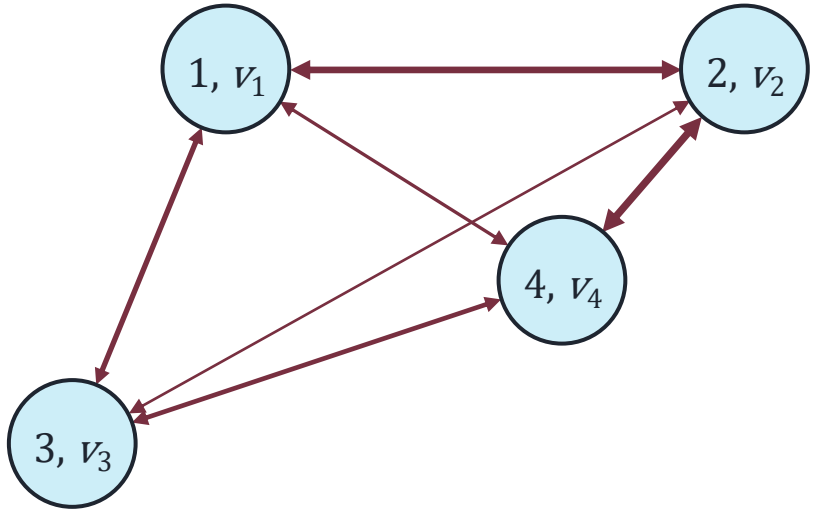
Fig. 1. The IceCube Neutrino Observatory with the in-ice array, its sub-

*N. Choma et al. IceCube collaboration, Graph Neural Networks for IceCube Signal Classification, arXiv:1809.06166v1

IceCube GNN : Data (2)

- IceCube data are the signals from n Digital Optical Modules (DOMs). The signals are modeled as the nodes of a graph.

- Each node is associated with a d-dimensional (row-wise) vector of attributes $\boldsymbol{v_i} = (x_1, \cdots, x_d)_i$, three of which are the spatial coordinates (x, y, z) of the associated digital optical module.



IceCube GNN: Data (3)

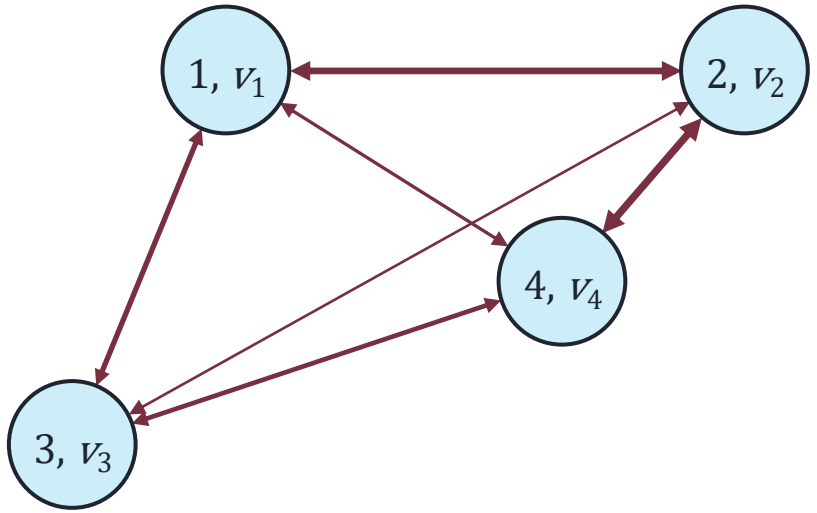
For an event with n signals, an $n \times n$ **adjacency matrix**, $\boldsymbol{A} \equiv \boldsymbol{E}$, is constructed using the spatial data only,

$$(\boldsymbol{A})_{\boldsymbol{ij}} = \mathrm{softmax}(d_{ij},\ \mathrm{dim} = 1),$$

where

$$d_{ij} = \exp\left(-\alpha \left\| x_i - x_j \right\|^2\right),\ \ \sum_{j=0}^{n-1} d_{ij} = 1$$

and α is a free parameter. The matrix $\boldsymbol{A}$ models the edges of the graph.

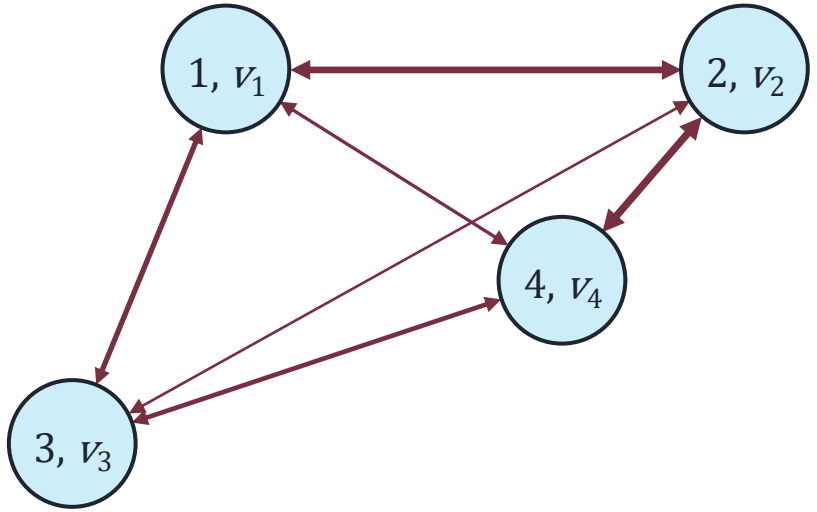


IceCube GNN: Data (4)

The vectors v_i are concatenated *vertically* into an $n \times d$ matrix: $V = [v_1; \cdots; v_n]$

$$V = \begin{pmatrix} x_1 & y_1 & z_1 & \cdots \\ x_2 & y_2 & z_2 & \cdots \\ \vdots & \vdots & & \vdots \\ x_n & y_n & z_n & \cdots \end{pmatrix}$$

(Note: horizontal concatenation is denoted by $V = [v_1, \cdots, v_n]$.)



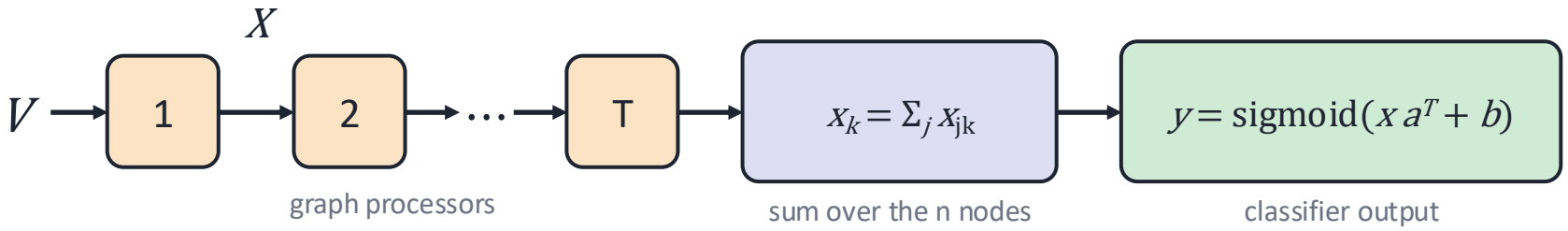
IceCube GNN: GNN (1)

- The $n \times d$ matrix, V, passes through a sequence of T graph processors, each with its own parameters.

- A graph processor is parameterized by a $2d \times d/2$ matrix of parameters, w, and a single scaler b and computes:

$$\boldsymbol{Y} = [\boldsymbol{A}V, \boldsymbol{V}]w + b\boldsymbol{I}$$

- The $\boldsymbol{Y}$ operation is an example of a graph convolution (**GNNConv**) and $\boldsymbol{I}$ is an $n \times d/2$ matrix of ones.

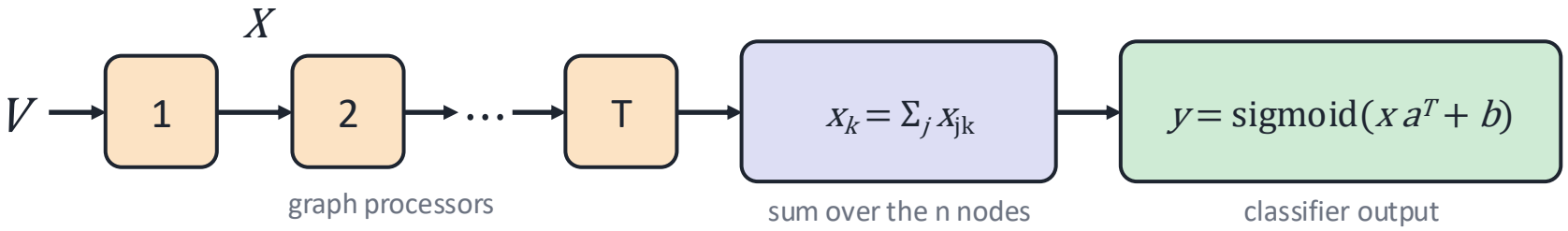


IceCube GNN: GNN (2)

- In IceCube, the vertices, V, of the output graph $G = (V, A)$ are constructed through *horizontal* concatenation as follows:

$$\boldsymbol{X} = [\mathrm{ReLU}(\boldsymbol{Y}), \boldsymbol{Y}]$$

with, as usual, the ReLU function applied *element-wise*.



IceCube GNN: GNNConv (1)

$$\boldsymbol{Y} = [\boldsymbol{AV}, \boldsymbol{V}]w + b,$$
$$\boldsymbol{X} = [\mathrm{ReLU}(\boldsymbol{Y}), \boldsymbol{Y}]$$

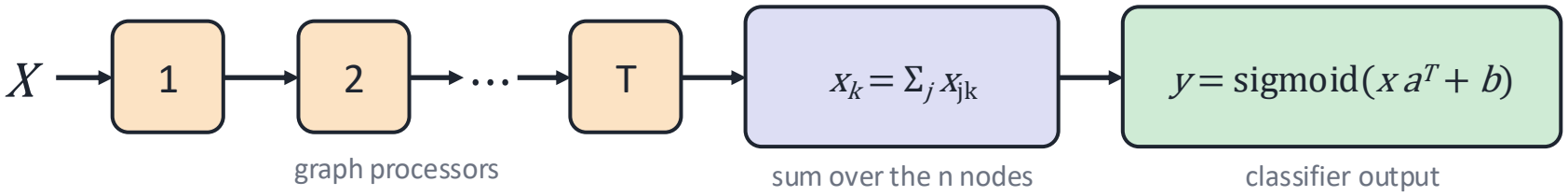
Implementation

```
A    = edges(V)
AV   = torch.matmul(A, V)
AVV  = torch.cat([AV, V], dim=1)
Y    = torch.matmul(AVV, w)  + b
X    = torch.cat([F.relu(Y), Y], dim=1)
```

IceCube GNN: GNN (3)

- At the end, $\boldsymbol{X}$ is mapped to a d-dimensional vector $\boldsymbol{x}$ using a *permutation invariant map* with respect to the nodes (vertices) and *invariant* with respect to the number of nodes.

- Finally, the vector $\boldsymbol{x}$ is mapped to the scalar output $0 \leq y \leq 1$ using a sigmoid.



IceCube GNN: GNN Results

The GNN is **6.3** times more efficient than the IceCube physics baseline analysis at a signal to noise ratio that is **3** times better.

It also outperforms a 3D CNN.

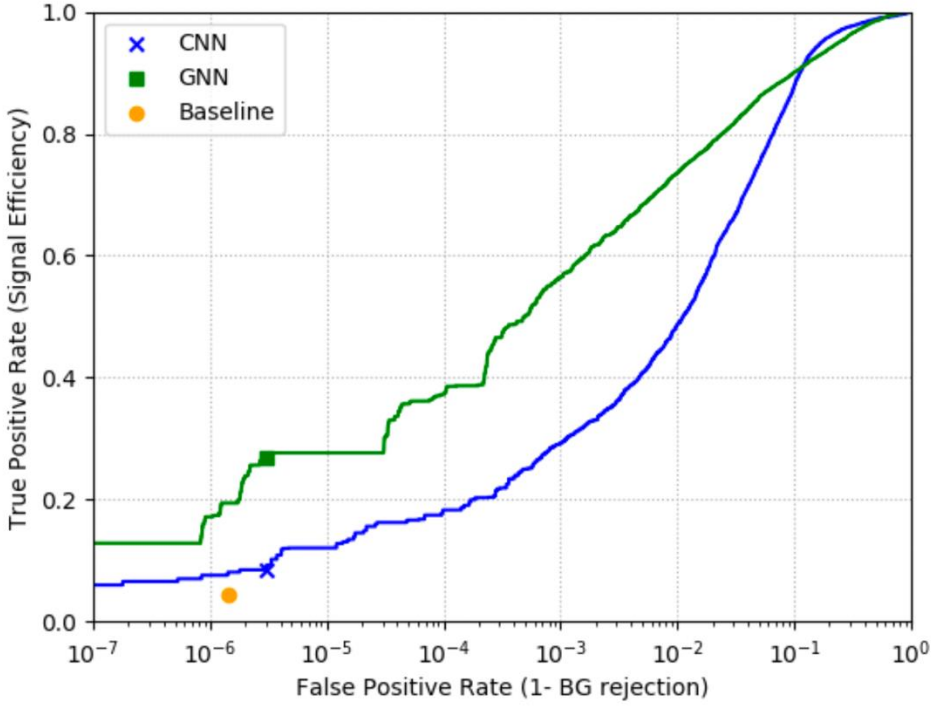


Fig. 3. Receiver operating characteristic curve for various methods considered in this paper. The green square and blue X indicate the evaluation point for the GNN and CNN, respectively.

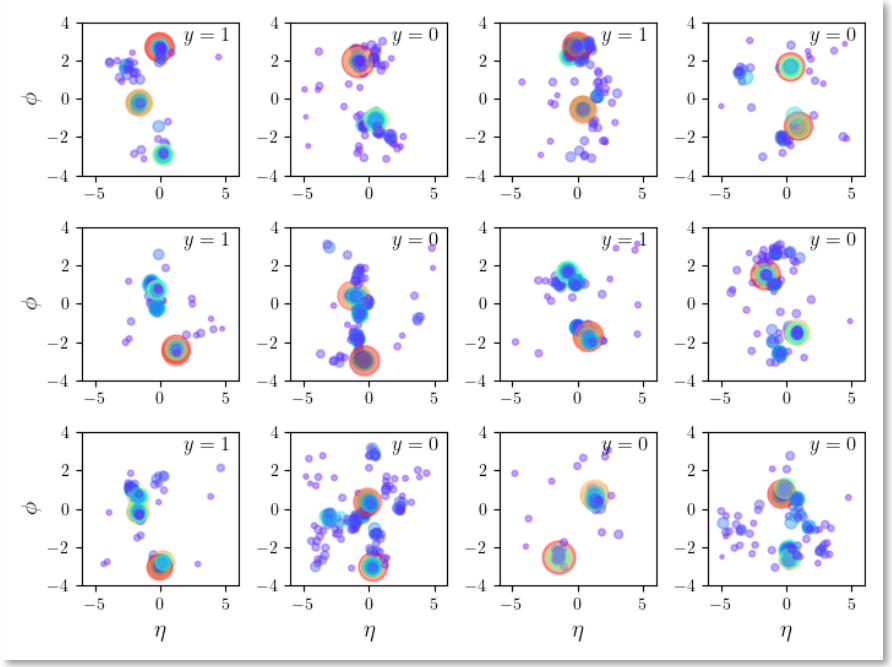
GNN TUTORIAL

GNN Tutorial: Goal

In this tutorial, we distinguish two classes of particle collision events: signal ($y = 1$) or background ($y = 0$), where each event is an array of particles with attributes (p_T, η, ϕ), the transverse momentum, i.e., the momentum in the $x-y$ plane, and the particle direction.

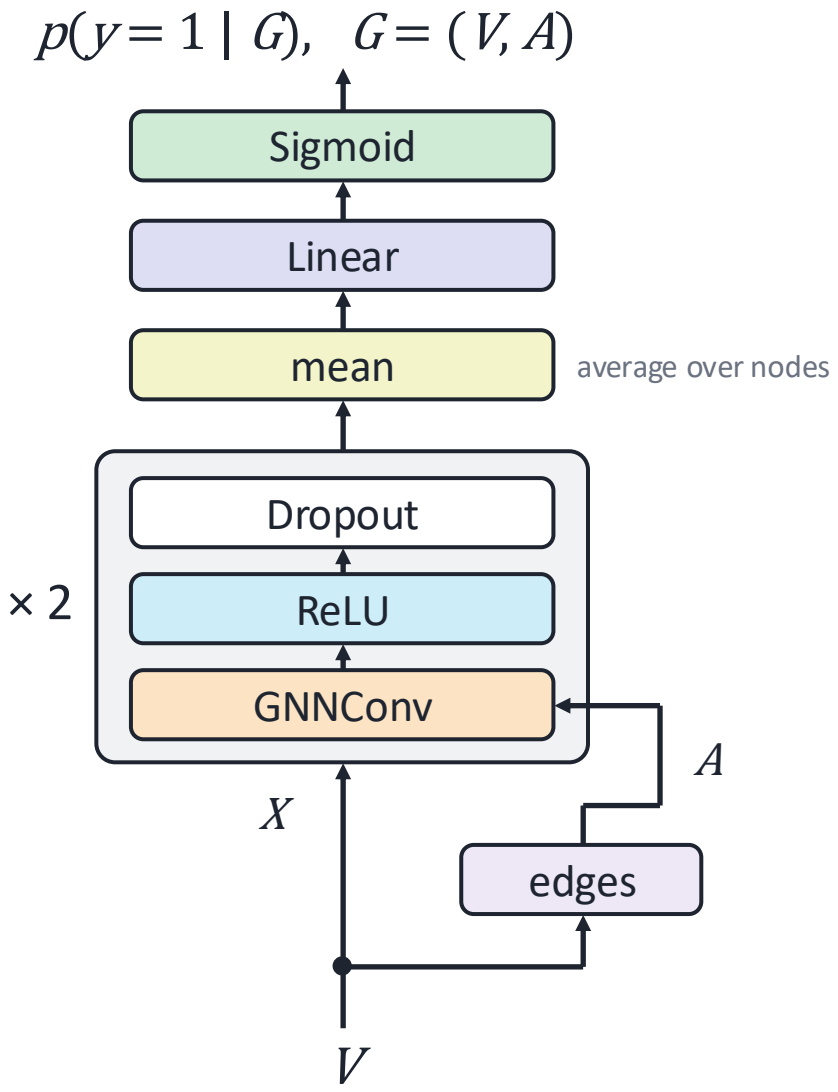
We use a simpler graph convolution than IceCube's:

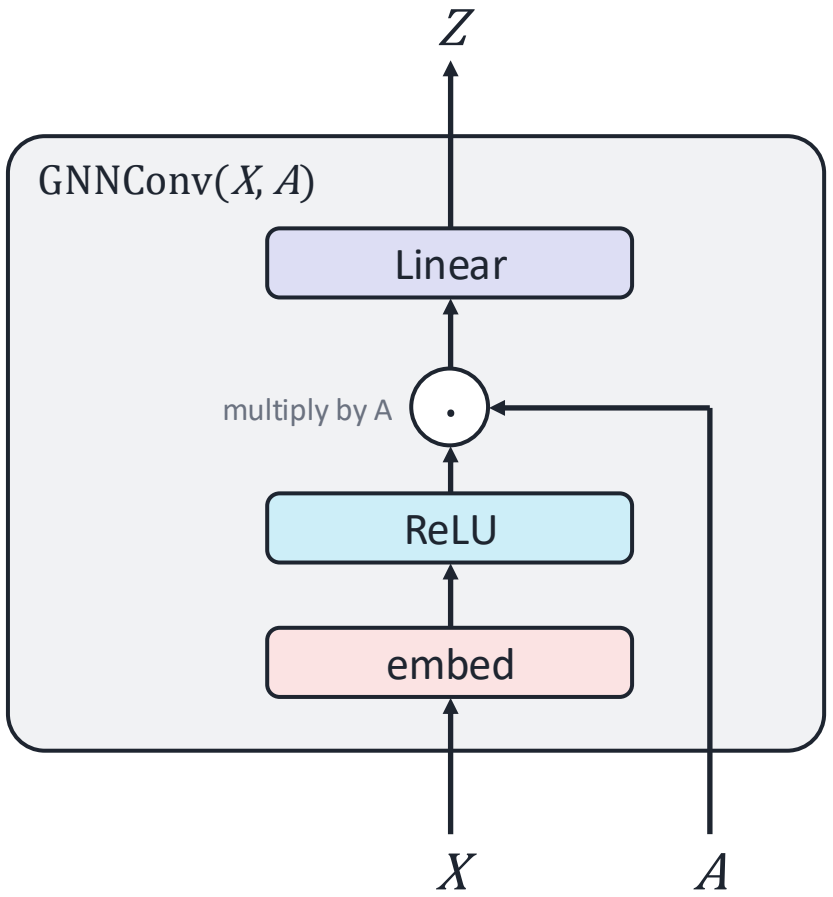
$$X = \sigma(XW_1^T + B_1) \quad \text{embed}$$
$$X' = AX$$
$$X'' = X'W_2^T + B_2$$



https://lhco2020.github.io/homepage/

GNN: Model





$$Z = \text{Linear}(A \cdot \text{ReLU}(\text{embed}(X)))$$

Summary

- Data instances in which the number of data points varies can be modeled with graphs $G = (V, E)$ using graph neural networks (GNN).

- We looked at a specific application of GNNs by IceCube, which, for this experiment, achieved state-of-the-art classification results.